



ISMAIL SK

Details

+91 8240691613

ismailsheikh8240691613@gmail.com

Konnagar, West Bengal, 712235

[GitHub](#) | [LinkedIn](#) | [Portfolio](#)

Language

- English — Fluent
- Hindi — Comfortable

Additional Activities

- Audio Story Creation

Summary

Full-stack AI Engineer crafting intelligent applications from scratch. Deep hands-on experience across modern web stacks, ML/DL, NLP, RAG pipelines, and Computer Vision. Focused on turning complex AI systems into production-grade, end-to-end software.

TECHNICAL SKILLS

- **AI/ML:** Machine Learning, Deep Learning, CNN, Computer Vision, RAG, ASR, LLM
- **Programming:** Python, JavaScript, TypeScript, Java, C, PostgreSQL
- **Frameworks & Tools:** TensorFlow, OpenCV, Scikit-learn, AWS, Next.js, React Native, Express.js, FastAPI, LangChain, RESTful APIs
- **Core CS:** OOP, DBMS, DSA
- **Others:** Data Visualization, Prompt Engineering

PROJECTS

- **Hybrid Face Spoofing Detection (Deep Learning + Local Descriptors)**
Developed a hybrid anti-spoofing model using YOLO features fused with LBP, LPQ, WLD, HOG, BSIF, and LTP; classified using SVC for improved robustness. | [Live](#)
- **Enterprise Document to Searchable Knowledge System (RAG-Based)**
Built a RAG-based system to convert enterprise documents into a searchable knowledge base using document embeddings, semantic retrieval, and Tesseract OCR for extracting text from scanned documents. | [Live](#)
- **DeepFakeShield**
Developed an AI-powered media verification system combining texture analysis with EfficientNet deep learning to detect AI-generated images and videos and provide reliable authenticity scores. | [Live](#)
- **Entertainment 24/7**
Built a full-stack movie platform using Next.js and FastAPI with JWT authentication, HTTP-only cookies, token refreshing, OTP-based password reset, user support, and an admin monitoring dashboard. | [Live](#)

INTERNSHIPS

- Intel Unnati Industrial Training 2025
- Infosys Springboard Internship 6.0
- TATA | Data Visualization: Empowering Business with Effective Insights.
- TATA | GenAI Powered Data Analytics Job Simulation

EDUCATION

- **B.Tech in Computer Science & Engineering (Specialization: Artificial Intelligence & Machine Learning)**
M.C.K.V Institute of Engineering | 2022 – 2026
Cumulative CGPA: 9.05
- **Senior Secondary (XII), WBCHSE**
Uttarpara Children's Own Home | 2022
Percentage: 81.2%
- **Secondary (X), WBBSE**
Uttarpara Children's Own Home | 2020
Percentage: 75.43%

AI Fresher – Technical Assignment

GitHub: [Click here to view the repository](#) | Live Demo: [Click here to view the project](#)

A. Problem Understanding

In a regular customer support environment, a large number of customers share their problems through support tickets. These tickets need to be categorized based on the type of issue so they can be handled appropriately. Although this categorization can be done manually, it can become inefficient when the number of tickets increases.

To address this problem, this project uses machine learning to automatically categorize customer support tickets based on their ticket descriptions. The system analyzes the text provided by the customer and predicts the most appropriate support category.

Additionally, the system includes an AI-assisted smart reply feature that uses the ticket description and predicted category to generate a suggested response with possible next steps for the customer.

B. Dataset

i. Dataset Source

Initially, I tried to find a suitable customer-support dataset on Kaggle. However, most of the available datasets did not match the structure required for this project. The project required a CSV dataset containing fields such as ticket_id, ticket_description, category, priority, and status, along with at least four different ticket categories.

A few datasets matched some of these requirements, but they were still missing one or more required fields. Adding the missing fields synthetically was possible, but some of the datasets contained a large number of records with relatively repetitive or unrelated ticket descriptions. Some descriptions were also repeated multiple times, which could make the dataset less suitable for evaluating a basic text-classification model.

Therefore, I decided to create a synthetic dataset specifically designed according to the project requirements. The dataset was generated using the create_dataset.ipynb notebook located in the project's notebooks folder. I created different ticket-description patterns for each category and generated approximately 20–30 different descriptions for each category. This provided a more controlled and diverse dataset for the classification task.

ii. Dataset Size and Categories

The final dataset contains **250 customer-support ticket records** distributed across five categories:

1. **Login Issue**
2. **Application Error**
3. **Report**
4. **Account Update**
5. **Performance**

The dataset contains a variety of ticket descriptions, and most individual descriptions occur only a small number of times. This provides more variation in the text used for training and testing the classification models.

iii. Dataset Fields

The dataset contains the following five fields:

Field	Description
ticket_id	Unique identifier for each support ticket
ticket_description	Description of the customer's problem or request
category	Category assigned to the ticket
priority	Priority level of the ticket, such as Low, Medium, High, or Critical
status	Current status of the ticket, such as Open, Closed, or In Progress

iv. Status Generation

Since the dataset was synthetically created, the status field was generated based on the ticket's priority. Different probabilities were assigned to each status depending on the priority level. For example, Critical tickets were given a higher probability of being Open, while Low priority tickets were more likely to be Closed. A small probability of In Progress was also maintained across the priority levels.

C. Data Preprocessing

Since the dataset was synthetically generated specifically for this project and contains only 250 records, the initial data-quality checks showed no missing values and no duplicate records. Therefore, no additional imputation or duplicate-removal process was required.

The ticket descriptions still required NLP-based text preprocessing before being used for model training. This was performed inside the machine-learning pipeline in train.ipynb. The preprocessing was applied to make the text more consistent and reduce unnecessary variations in the input data.

The following basic text-cleaning steps were performed:

- Converted all text to lowercase.
- Removed special characters and unnecessary symbols.
- Removed leading and trailing whitespace, which was handled by the TF-IDF.
- English stop words were handled by the TF-IDF vectorizer during feature extraction.

The main text cleaning done in the pipeline was:

```
model_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(
        lowercase=True,                # Step 1: Lowercases text
        token_pattern=r"(?u)\b[a-zA-Z]+\b", # Step 2: Keeps letters only (drops numbers & symbols)
        stop_words="english"           # Step 3: Removes English stop words & whitespace
    )),
    ("classifier", MultinomialNB())
])
```

This preprocessing was required because the ticket descriptions are the primary input to the classification model. Cleaning the text helps provide a more consistent representation of the same or similar words and removes unnecessary characters that could introduce noise into the TF-IDF features.

Key Dataset Observations

During the exploratory analysis, several important observations were identified:

- **Dataset Size & Diversity:** The dataset contains 250 records with a high level of diversity in ticket descriptions. Most descriptions occur only a few times, with no single description being excessively repeated. This provides varied textual examples across categories and reduces reliance on exact duplicate descriptions during model training.
- **Priority Distribution:** The tickets contain four priority levels: Critical, High, Medium, and Low, with Medium and High priority tickets forming the majority.
- **Status Variance:** Ticket status is distributed across Open, Closed, and In Progress, with the distribution varying according to ticket priority.
- **Category & Priority Distribution:** The Priority $\times$ Category analysis shows that all five categories contain tickets across the different priority levels, providing variety in the dataset.

Data Quality: No missing values or duplicate records were found. Therefore, no additional data-cleaning operations were necessary before the NLP preprocessing used for model training.

D. Model Selection

For this project, three classification algorithms were selected for training and performance comparison: Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Machine (SVM).

The main input feature used for classification was ticket_description, while category was used as the target variable. Since the ticket descriptions are text-based, the text was converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) before being provided to the classification models.

i. Multinomial Naive Bayes

Multinomial Naive Bayes was selected because it is a commonly used algorithm for text classification. It works well with text data represented using word or term frequencies, making it suitable for customer-support ticket classification. It is also computationally efficient and relatively simple compared with more complex classification approaches.

The Multinomial variant was specifically selected because the input consists of textual documents represented through TF-IDF features. It is designed to work with features that represent the occurrence and importance of terms within documents, making it a suitable choice for this type of ticket-classification problem.

ii. Logistic Regression

Logistic Regression was selected as another baseline classification model because it is widely used for text classification when combined with TF-IDF features. It can handle a high-dimensional feature space and provides a useful comparison against Naive Bayes.

iii. Linear SVM

Linear SVM was selected because it is also well suited to text classification problems with high-dimensional TF-IDF features. It provides another strong traditional machine-learning approach for comparing the classification performance on the same dataset.

Final Model Selection

All three models were trained and evaluated using the same TF-IDF-based feature representation and the same ticket-classification task. While all three models produced useful results, Multinomial Naive Bayes provided slightly better optimized performance on this dataset. Therefore, Multinomial Naive Bayes was selected as the final classification model.

The detailed performance comparison between the three models is presented in the Results section. This comparison was used as the practical basis for the final model selection rather than selecting an algorithm based only on theoretical advantages.

E. Training and Testing

i. Training and Testing Split

The dataset was divided into 80% training data and 20% testing data using `train_test_split()`. The training data was used to train the models, while the testing data was kept separate and used later to evaluate how well the trained models performed on unseen ticket descriptions.

A `random_state` of 42 was used to make the split reproducible. `stratify=y` was also used to maintain a similar proportion of all five ticket categories in both the training and testing sets.

ii. Training Process

The main training process was built using a machine-learning pipeline. The pipeline combines text preprocessing, TF-IDF feature extraction, and the classification model into a single workflow.

For the final model, Multinomial Naive Bayes was used as the classifier. Since the input is completely text-based, TF-IDF was used to convert the ticket descriptions into numerical features that could be processed by the model.

To improve the model's performance and reduce the possibility of overfitting, hyperparameter tuning was performed using `GridSearchCV`. The main focus of the tuning was the Multinomial Naive Bayes classifier, while some TF-IDF parameters were also varied.

The following parameters were tested:

- **`classifier__alpha`** — This is the main parameter for Multinomial Naive Bayes. We tested different values: 0.1, 0.5, 1.0, 2.0, and 5.0. Alpha controls smoothing, which prevents the model from assigning a probability of zero to words that are not seen in a particular category. Higher values apply more smoothing and can help reduce overfitting.
- **`tfidf__max_df`** — Controls how common a word can be before it is removed. If a word appears in too many tickets, it may not be useful for distinguishing between categories, so very common words can be ignored.
- **`tfidf__min_df`** — Controls how rare a word can be before it is removed. Words that appear in only a very small number of tickets may be noise or typing variations, so removing them can make the model more reliable.
- **`tfidf__max_features`** — Sets a maximum limit on the number of words or text features used by TF-IDF. This keeps the feature set manageable and prevents unnecessary features from being used.
- **`tfidf__ngram_range`** — We tested both single words (unigrams) and single words + two-word combinations (bigrams). For example, "password" is a unigram, while "forgot password" is a bigram. This allows the model to determine whether individual words or short word combinations provide better classification results.

- **tfidf_sublinear_tf** — We tested both True and False. When enabled, TF-IDF reduces the impact of words that appear many times in the same ticket. This can prevent frequently repeated words from having too much influence on the model.

GridSearchCV tested different combinations of these parameter values using 5-fold cross-validation. The combinations were evaluated using macro F1-score, which gives equal importance to each ticket category. The best-performing combination was then selected as the final configuration.

This approach allowed the Multinomial Naive Bayes model to be selected based not only on its default settings, but also on a tuned configuration that was better suited to the ticket-classification dataset.

iii. Testing Process

After the training and hyperparameter-tuning process was completed, the selected model was evaluated using the 20% testing dataset that had not been used for training.

The model's performance was measured using accuracy, precision, recall, F1-score, and a confusion matrix. These results were then compared with the results from Logistic Regression and Linear SVM.

For a new ticket, the trained pipeline automatically performs the required text preprocessing and TF-IDF transformation before passing the processed text to the trained Multinomial Naive Bayes classifier for category prediction.

F. Results

The three selected classification models — Multinomial Naive Bayes, Logistic Regression, and Linear SVM — were trained and evaluated using the same dataset, TF-IDF features, and testing data. Their performance was compared using accuracy, precision, recall, and F1-score.

i. Model Performance Comparison

Model	Training Accuracy	Test Accuracy	Precision	Recall	F1 Score
Multinomial Naive Bayes	99.50%	98.00%	0.98	0.98	0.98
Logistic Regression	99.97%	98.00%	0.98	0.98	0.98
Linear SVM	99.98%	98.00%	0.98	0.98	0.98

All three models achieved the same **98% test accuracy** and the same **macro precision, recall, and F1-score of 0.98**. This indicates that the three models performed very similarly on the test dataset.

ii. Meaning of the Evaluation Metrics

- **Accuracy:** Percentage of total tickets that were classified correctly.
- **Precision:** How often the predicted category was actually correct.
- **Recall:** How many tickets belonging to a category were correctly identified.
- **F1 Score:** A combined measure of precision and recall.

iii. Confusion Matrix Analysis

The confusion matrices generated for Multinomial Naive Bayes, Logistic Regression, and Linear SVM were identical. Therefore, a single representative confusion matrix is shown below instead of repeating the same figure for each model.

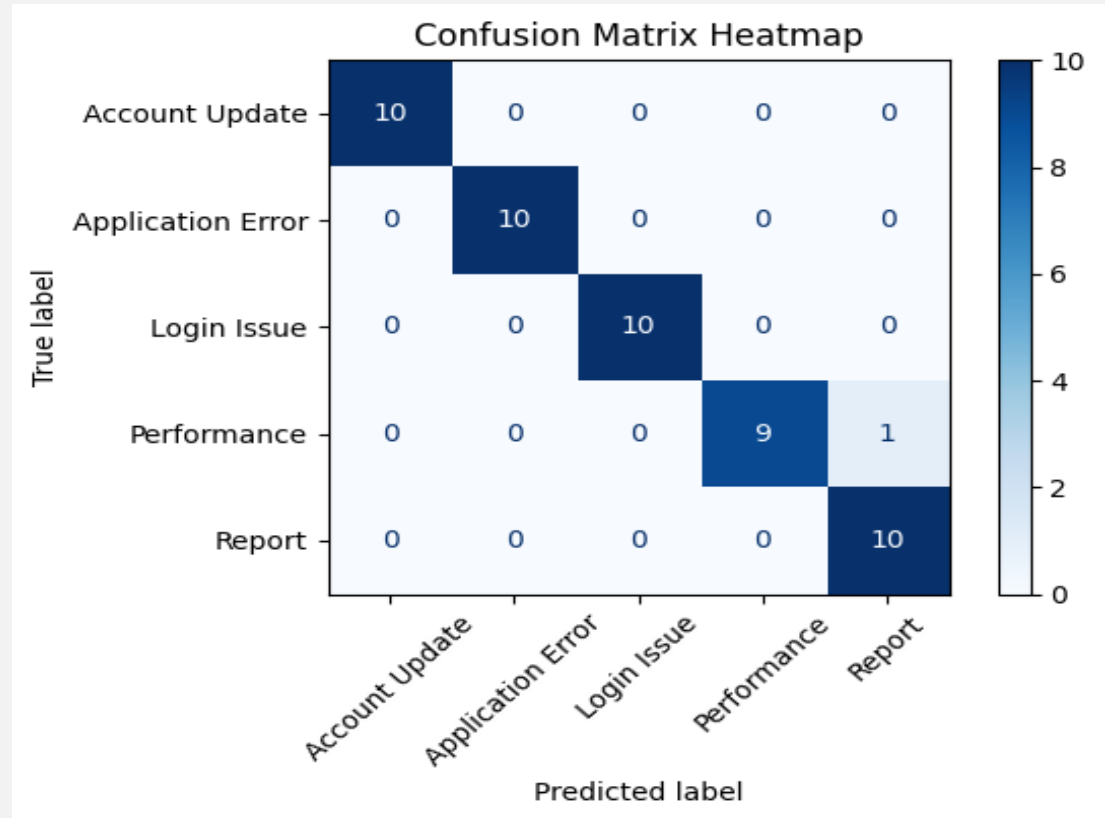


Fig. 1: Confusion Matrix

The matrix shows that 49 out of 50 test tickets were classified correctly. All tickets in the Account Update, Application Error, and Login Issue categories were classified correctly. The Report category was also correctly classified. The only error occurred when one Performance ticket was classified as Report (Fig. 1). This indicates that the main classification confusion in the test set was between the Performance and Report categories.

Final Model Selection

Although all three models achieved the same test accuracy and evaluation scores, Multinomial Naive Bayes was selected as the final model. Its training accuracy was 99.50%, compared with 99.97% and 99.98% for Logistic Regression and Linear SVM, while all three achieved 98% test accuracy.

Multinomial Naive Bayes was also selected because the problem is entirely text-based, and this algorithm is well suited to text classification with TF-IDF features. It is relatively simple, computationally efficient, and appropriate for the size and nature of this project.

Therefore, the final selection was based on both the observed model performance and the suitability of Multinomial Naive Bayes for the ticket-description classification task.

G. Prediction

i. Ticket Category Prediction

After comparing the three classification models, Multinomial Naive Bayes was selected as the final model. The trained model was then used to classify new ticket descriptions that were not part of the training process.

For each test input, the system processes the ticket description through the same NLP preprocessing and TF-IDF transformation used during training. The trained Multinomial Naive Bayes model then predicts the most likely ticket category.

The following screenshots (Fig. 2) demonstrate sample predictions made by the final model.

	Test Input	Expected Category	Predicted Category
0	I can access the website, but it won't let me ...	Login Issue	Login Issue
1	Everything was working earlier, but the system...	Application Error	Login Issue
2	I need a summary of our transactions for the p...	Report	Report
3	Some of the information on my profile is outda...	Account Update	Account Update
4	The system eventually responds, but every acti...	Performance	Performance
5	I have tried several times, but I still cannot...	Login Issue	Account Update
6	I get an unexpected message after submitting t...	Application Error	Application Error
7	Could you provide the figures for our sales ac...	Report	Report
8	Our administrator needs to make some changes t...	Account Update	Account Update
9	Opening customer information has become notice...	Performance	Performance
10	The credentials I normally use are no longer a...	Login Issue	Login Issue
11	The request reaches the server but something g...	Application Error	Account Update
12	I need a document containing the activity reco...	Report	Report
13	I recently changed my contact details and want...	Account Update	Account Update
14	Searching through customer records now takes s...	Performance	Performance

Fig. 2: Model predictions on the test dataset.

ii. AI-Assisted Customer Response

As an additional feature, the system uses an LLM-based customer response component through the Groq API. After the machine-learning model predicts the ticket category, the system provides the original ticket description and predicted category to the LLM. The LLM generates a short, professional, and customer-friendly response based on the available information. The response is focused on providing quick solutions or practical guidance related to the customer's issue rather than generating a random or unrelated response. It also avoids inventing unsupported information such as company policies, refunds, account details, or other facts that are not provided in the ticket.

The system uses openai/gpt-oss-120b through the Groq API. Groq was selected because its inference service provides fast response generation, which is useful for the interactive Streamlit application. Gemini was also considered during development, but Groq was selected because its response time was more suitable for the interactive nature of the application.

H. Limitations

Although the system achieved good results on the available test data, the current solution has some important limitations.

1. Synthetic Dataset

The dataset was created specifically for this project, so the ticket descriptions follow a controlled set of writing patterns. A model trained on this type of data may not generalize well to real customer-support tickets, where users can have very different writing styles, vocabulary, sentence structures, and ways of describing the same issue.

2. Limited Training Data

The dataset contains only 250 records. This is sufficient for demonstrating basic NLP and machine-learning concepts, but it provides a limited number of examples for the model to learn from. A real customer-support system would require a much larger collection of tickets to represent a wider range of possible issues and descriptions.

3. Limited Categories

The system currently supports only five categories: Account Update, Application Error, Login Issue, Performance, and Report. Real customer-support systems may need to handle many more types of issues. However, simply adding more categories would not be sufficient; each category would also require enough diverse training examples to allow the model to learn meaningful patterns.

4. Prediction Confidence and Unfamiliar Wording

The model uses TF-IDF and Multinomial Naive Bayes. Unlike an LLM, it does not understand the complete meaning of a sentence. Instead, it relies mainly on the words and text features learned from the training data to calculate probabilities for each category.

For example, a prediction may produce:

Account Update	18.0%
Application Error	21.0%
Login Issue	26.7%
Performance	17.0%
Report	17.3%

In this case, Login Issue is selected because it has the highest probability, but the relatively close probabilities indicate that the model has limited confidence in the prediction.

This can happen when a new ticket uses wording that is different from the patterns present in the synthetic training data. For example, the training data may contain:

"I cannot sign in to my account."

while a real customer may write:

"I can access the website, but it won't let me get past the sign-in screen."

A human can recognize that both describe a login problem, but TF-IDF primarily relies on learned text features rather than understanding the meaning of the two sentences. Therefore, unfamiliar wording can result in a lower confidence score even when the predicted category is correct.

5. Real-World Performance Not Established

The 98% test accuracy demonstrates how the model performed on the project's held-out test dataset. However, this result should not be considered a benchmark for real-world customer-support classification. The test data was created from the same synthetic dataset, so it does not fully represent the variety of language found in actual customer interactions. Evaluation on a larger collection of genuine customer-support tickets would be required to determine how well the system generalizes to real-world use.

I. Future Improvements

If this system were developed for a real company, several improvements could be made to make it more reliable, scalable, and useful in a real customer-support environment.

- **Larger and Real-World Dataset:** The system should be trained on a much larger collection of real customer-support tickets. This would expose the model to different writing styles, vocabulary, spelling mistakes, and ways of describing the same problem.
- **More Categories:** The classification system could be expanded to support more specific categories and subcategories as required by the company's support workflow.
- **Improved Text Preprocessing:** More advanced preprocessing could be introduced to handle spelling mistakes, abbreviations, noisy text, short messages, and other variations commonly found in real customer messages.
- **Model Improvement:** Different machine-learning and NLP approaches could be evaluated as the dataset grows. The model could be periodically re-evaluated to determine whether a more suitable approach provides better results.
- **Deep Learning or LLM-Based Classification:** With sufficient data and resources, deep-learning models or LLM-based approaches could be explored for understanding more complex and context-dependent customer descriptions.
- **Continuous Model Retraining:** New customer-support tickets could be regularly added to the training dataset so that the model can adapt to newly occurring issues and changes in customer language.
- **Human Feedback:** Support agents could review incorrect or uncertain predictions. Their corrections could be collected as feedback and used to improve future versions of the model.
- **Production API:** The trained model could be deployed through a production API so that it can be integrated with an existing customer-support application or ticketing system.
- **Database Integration:** Ticket information, predicted categories, model responses, and agent corrections could be stored in a database. This would also provide a continuous source of real-world data for monitoring and future model retraining.

- **Confidence-Based Human Review:** Tickets for which the model has low prediction confidence could be automatically sent to a support agent instead of relying completely on the model. This would provide an additional safety layer for real-world usage.

J. Use of AI Tools

AI tools were used primarily as a learning and research aid throughout the project. They helped me understand underlying concepts related to machine learning, NLP, model evaluation, and hyperparameter tuning, as well as discuss good development practices and ways to maintain a professional, industry-oriented approach as a fresher.

The technical decisions, implementation choices, model selection, evaluation, testing, and final conclusions were made by me. AI was used to support my understanding and improve the quality and professionalism of the development process, rather than to make decisions on my behalf.

K. Project Links

- GitHub Repository: https://github.com/Ismail007-Sk/AI_Assignment_Ismail_Sk.git
- Live Application: <https://aiassignmentismailsk-pbuwv7x6trtqeccye53n.streamlit.app/>